

The Reconnection: A Worked Example of Custody Discipline After a Hiatus

3 August 2026 · Ken Tombs with Claude (design lead, Opus 4.8) · A close reading of the Session 7 wake-up — the pilot's first reconnection after a genuine gap (17 days, 15 July – 3 August). The frame is deliberately left open: the seven events are tracked faithfully and each is read for what it demonstrably shows, with the wider patterns explored rather than declared. Companion to the Session 7 log (Entries 082–089).

Why this note exists

For six sessions the pilot asserted a founding doctrine — the chat is a convenience, the document is custody — but never truly tested it, because every session boundary had been a matter of hours. Session 7 opened after seventeen days. Neither operator nor design lead held the workspace's state in working memory; whatever we knew had to be reconstructed from records written a fortnight earlier. The wake-up ritual therefore stopped being a routine and became an experiment: could a verified working state be rebuilt from artefacts alone, across a gap long enough to simulate a real adjournment?

It could, and it did — but the value is not in the headline. It is in the SEQUENCE. Seven distinct anomalies arose during the reconnection, and the way each was met, mistaken, tested, and resolved is a more honest picture of custody discipline in practice than any procedure document. This note tracks all seven, because the pattern they form together may be the most transferable thing the pilot has produced.

The seven events

Event 1 — The stale-transcript worry

What happened. Having started the bench transcript, the operator paused before proceeding and asked: could two transcripts be running at once, given a previous-dated one had been started? Rather than assume, we checked the old transcript's last-write time — 15 July, seventeen days cold — which proved no live window was touching it.

What it shows. The operator's first instinct after a gap was SUSPICION, not assumption, and the suspicion was resolved by an artefact (a timestamp) rather than a memory. This is the doctrine working at the smallest scale: the question "is something still running from before?" is answered by the filesystem, not by recollection of what was shut down. A dormant 22 KB file also silently confirmed the Session 6 bench record had survived the hiatus intact — the first small proof of custody continuity, obtained as a side effect of disproving a worry.

Event 2 — The bench parked inside the corpus

What happened. The bench's prompt read `...\enron_mail_20150507\maildir>` — two levels deeper than the ritual specifies, inside the evidence tree itself. Cause: PowerShell had been launched from an Explorer address bar while browsing the corpus, so the window opened wherever the operator had been looking, not at a neutral home directory.

What it shows. Launch CONTEXT silently determines position, and the trap is invisible until the prompt path is read aloud. Over a seventeen-day gap the muscle memory that would once have caught this had faded — which is precisely why the ritual's "read the path before proceeding" rule exists, and precisely when it earns its keep. The fix was trivial (walk back out); the lesson was not: a bench parked in the evidence tree is the exact precondition of the pilot's worst prior contamination (the truncated copy of Session 3). The procedures gain an amendment — launch the bench from the Start menu, never an

address bar — but the deeper point is that geometry accidents recur whenever discipline goes cold, and only reading, not remembering, catches them.

Event 3 — The "missing" outputs folder

What happened. The -Force workspace listing appeared not to show pilot_outputs. The operator provisionally attributed the loss to an operator error in a prior task, and this was logged as such.

What it shows. Here the discipline was, for a moment, ABOUT to fail — not by missing a fault but by MANUFACTURING one. A provisional human-error attribution was recorded on the strength of a single reading. This is the more interesting failure mode, and the rarer one to guard against: the navigable-chain method (grade every inference, invite an artefact to confirm or kill it) exists as much to prevent inventing faults as to prevent missing them. The attribution was correctly held as provisional — which is what allowed the next event to overturn it cleanly.

Event 4 — The mkdir collision that overturned the attribution

What happened. Recreating the "lost" folder, mkdir reported it already existed. A direct listing then showed it present and empty — exactly as left at Session 6 close. The provisional operator-error attribution was retracted; no loss had occurred.

What it shows. The direct check overruled the earlier reading, and the record self-corrected FORWARD — the retraction logged beside the provisional claim, neither erased. The lesson is procedural and sharp: verify an anomaly by direct check before recording its cause. The initial reading was the error, not the folder. It is worth sitting with how close this came to entering the permanent record as "operator lost the outputs folder during the hiatus" — a plausible, self-blaming, and entirely false finding, prevented only because the attribution was held provisional and a two-second check was run before it hardened.

Event 5 — The home-directory trust prompt

What happened. The subject was launched with claude, and Claude Code asked to trust C:\Users\ktomb — the entire user profile — because the launch window was parked at the home directory, not the workspace. The operator caught it and exited without granting.

What it shows. This is Entry 002 of Session 1, returning seventeen days later, identical in every respect: a launch from the wrong directory, the tool offering the broadest possible trust, one keystroke from granting an AI read/edit/execute over everything under the user profile. The recurrence is the finding. The same trap reappeared the moment discipline was cold, and was caught by the same defence — reading the path the prompt named, rather than reflexively confirming. Trust granted here would have undone the entire scoped-boundary architecture in a single click. That the pilot's oldest hazard returned intact after a gap suggests these are not one-time lessons that stay learned; they are standing conditions that must be defended every session.

Event 6 — The Opus 4.8 default

What happened. Once launched from the correct floor, the subject came up on Opus 4.8 — not the pinned Sonnet 5. Investigation (via /model) showed the vendor's roster had changed over the hiatus (Opus 4.8 and Fable 5 arrived; Opus 4.7 gone) and the DEFAULT had silently moved to Opus 4.8, beyond the reach of the local DISABLE_UPDATES freeze.

What it shows. This one was neither operator error nor a phantom — it was real drift, and it mapped the freeze's exact boundary. The frozen-subject configuration held the client binary (v2.1.209, unmoved across seventeen days) and the entire filesystem footprint, but could not hold the model roster or default, because those live on the vendor's servers. A pinned client is a stable interface to a shifting roster. Every launch since the roster changed would silently have run on Opus unless caught — and it was caught only by reading the model name in the banner, a check the ritual now makes mandatory.

The freeze is real but bounded; the boundary runs exactly where local control ends and the vendor's service begins.

Event 7 — The re-pin and the bounded verdict

What happened. The subject was manually re-pinned to Sonnet 5 (still offered, promo through 31 August — continuity preserved); the banner confirmed v2.1.209, Sonnet 5. The subject, having served only to yield the freeze reading on a bench-and-drafting day, was then exited.

What it shows. The verdict of the whole hiatus, and it is stronger for being bounded rather than absolute: client and filesystem frozen exactly; roster and default drifted server-side and corrected by hand. VERIFICATION, not the freeze, is what guarantees the subject's identity at any given session. "We achieved perfect stability" would invite the first reviewer to find the exception; "we achieved bounded stability, mapped its exact boundary, and built verification to cover the remainder" already contains its own counterexample and cannot be so undone.

The pattern — explored, not declared

Three threads run through all seven events. The first is the clearest and hardest to argue with: EVERY anomaly was human or vendor in origin — an operator misreading, a launch-context accident, a server-side roster change — and NONE was the subject. Across seven reconnection events and six prior sessions, the AI has still never once damaged the workspace or misrepresented the evidence unchallenged. The disturbances come from the humans operating it and the vendor supplying it. If the pilot has a single most robust empirical finding, it may be this asymmetry, and the reconnection concentrated it into one morning.

The second thread: every anomaly was caught by reading an ARTEFACT rather than trusting a memory or an assumption — a timestamp, a prompt path, a folder listing, a banner. After a genuine gap, the architecture did not rely on anyone remembering; it relied on the disk, the clock, and the screen. This is the founding doctrine not asserted but demonstrated, under precisely the conditions it was built for. The chat did not carry the pilot across the gap. The records did.

The third thread is the one still open, and worth naming as open. Several of these traps were not new — the wrong-floor launch and the over-broad trust prompt are Session 1's hazards, returning intact. This suggests something uncomfortable and important: custody discipline may not consist of lessons that, once learned, stay learned. It may instead be a set of standing conditions that must be actively defended every single session, because the traps are properties of the tool and the environment, not mistakes the operator can permanently outgrow. If that reading holds, it changes what a procedure IS — not a training aid to be internalised and set aside, but a permanent instrument, run in full every time precisely because familiarity is not protection. The pilot does not yet have enough reconnections to know whether this is true. It is the question the next hiatus should be designed to answer.

Provisional conclusion

The reconnection worked: a verified state was rebuilt from records alone across seventeen days, four phantom anomalies were dissolved, one real drift was caught and corrected, and the freeze's boundary was mapped. But the worked example teaches more than the outcome. It shows custody discipline as a live practice — suspicion before assumption, artefact before memory, provisional before permanent, verification before trust — and it raises, without yet resolving, the possibility that this practice is not a phase the operator passes through but a discipline the work permanently requires. For a profession

that will one day place AI at the evidence table, that distinction is not academic. It is the difference between training people once and building instruments that protect them always.